

Cours Complet : Langage SQL (Structured Query Language)

Le Structured Query Language (SQL) est le langage standard et le plus répandu pour la gestion et la manipulation des bases de données relationnelles. Il permet d'interroger, d'insérer, de modifier et de supprimer des données, ainsi que de gérer la structure des bases de données elles-mêmes.

1. Introduction aux Bases de Données Relationnelles et SQL

Une base de données relationnelle organise les données en tables (ou relations), où chaque table est composée de lignes (enregistrements) et de colonnes (attributs). SQL fournit un moyen puissant et flexible d'interagir avec ces tables.

Les principales catégories de commandes SQL sont :

- **DDL (Data Definition Language)** : Pour définir la structure de la base de données (CREATE, ALTER, DROP TABLE).
- **DML (Data Manipulation Language)** : Pour manipuler les données (SELECT, INSERT, UPDATE, DELETE).
- **DCL (Data Control Language)** : Pour gérer les permissions (GRANT, REVOKE).
- **TCL (Transaction Control Language)** : Pour gérer les transactions (COMMIT, ROLLBACK).

2. Les Jointures (JOIN)

Les **jointures** sont des opérations fondamentales en SQL qui permettent de combiner des lignes de deux ou plusieurs tables en fonction d'une colonne commune ou d'une condition de jointure. Elles sont essentielles pour récupérer des informations réparties sur plusieurs tables liées entre elles.

2.1. Types de Jointures

Il existe plusieurs types de jointures, chacun ayant un comportement spécifique quant à la manière dont les lignes sont combinées et gérées en cas de non-correspondance :

- **INNER JOIN (Jointure Interne)** :
 - **Description** : Retourne uniquement les lignes qui ont des correspondances dans *les deux* tables. Les lignes sans correspondance dans l'une ou l'autre des tables sont

exclues du résultat.

- **Quand l'utiliser** : Lorsque vous avez besoin de données complètes et correspondantes des deux côtés de la jointure.
- **Exemple** :

SQL

```
SELECT etudiants.nom, cours.titre
FROM etudiants
INNER JOIN inscriptions ON etudiants.id = inscriptions.etudiant_id
INNER JOIN cours ON inscriptions.id_cours = cours.id;
```

Ce code SQL récupère les noms des étudiants et les titres des cours auxquels ils sont inscrits, en s'assurant que l'étudiant et le cours existent dans leurs tables respectives et qu'il y a une inscription correspondante.

- **LEFT JOIN (ou LEFT OUTER JOIN) (Jointure Gauche) :**
 - **Description** : Retourne toutes les lignes de la table de gauche (la première table mentionnée dans la clause `FROM`) et les lignes correspondantes de la table de droite. Si aucune correspondance n'est trouvée dans la table de droite, les colonnes de la table de droite contiendront des valeurs `NULL` .
 - **Quand l'utiliser** : Lorsque vous voulez inclure toutes les données de la table de gauche, même si elles n'ont pas de correspondance dans la table de droite.
- **RIGHT JOIN (ou RIGHT OUTER JOIN) (Jointure Droite) :**
 - **Description** : Similaire à la `LEFT JOIN` , mais retourne toutes les lignes de la table de droite et les lignes correspondantes de la table de gauche. Les colonnes de la table de gauche contiendront `NULL` si aucune correspondance n'est trouvée.
 - **Quand l'utiliser** : Lorsque vous voulez inclure toutes les données de la table de droite, même si elles n'ont pas de correspondance dans la table de gauche.
- **FULL JOIN (ou FULL OUTER JOIN) (Jointure Complète) :**
 - **Description** : Retourne toutes les lignes lorsqu'il y a une correspondance dans l'une des tables. C'est la combinaison de `LEFT JOIN` et `RIGHT JOIN` . Les valeurs `NULL` apparaissent là où il n'y a pas de correspondance.
 - **Quand l'utiliser** : Lorsque vous avez besoin de toutes les données des deux tables, qu'elles aient une correspondance ou non.

2.2. Jointures Multiples

Il est courant de joindre plus de deux tables. Pour ce faire, on enchaîne simplement les

clauses `JOIN` :

SQL

```
SELECT A.col1, B.col2, C.col3
FROM TableA AS A
JOIN TableB AS B ON A.id = B.id_a
JOIN TableC AS C ON B.id = C.id_b;
```

3. Fonctions d'Agrégation et Groupement (GROUP BY)

Les **fonctions d'agrégation** effectuent un calcul sur un ensemble de valeurs et retournent une seule valeur récapitulative. Elles sont souvent utilisées avec la clause `GROUP BY` pour effectuer ces calculs sur des groupes de lignes.

3.1. Fonctions d'Agrégation Courantes

Voici les fonctions d'agrégation les plus utilisées en SQL :

- `COUNT()` : Compte le nombre de lignes. `COUNT(*)` compte toutes les lignes, `COUNT(colonne)` compte les lignes où `colonne` n'est pas `NULL`.
- `SUM(colonne)` : Calcule la somme des valeurs numériques d'une colonne.
- `AVG(colonne)` : Calcule la moyenne des valeurs numériques d'une colonne.
- `MAX(colonne)` : Retourne la valeur maximale d'une colonne.
- `MIN(colonne)` : Retourne la valeur minimale d'une colonne.

3.2. La Clause GROUP BY

La clause `GROUP BY` est utilisée pour regrouper les lignes qui ont les mêmes valeurs dans une ou plusieurs colonnes en un seul groupe. Les fonctions d'agrégation sont ensuite appliquées à chacun de ces groupes.

- **Syntaxe de base :**

SQL

```
SELECT colonne1, fonction_agregation(colonne2)
FROM nom_table
GROUP BY colonne1;
```

- **Exemple :** Calculer le nombre d'employés par département.

SQL

```
SELECT departement, COUNT(*)  
FROM employes  
GROUP BY departement;
```

Ce code regroupe les employés par leur département et compte le nombre d'employés dans chaque département.

3.3. La Clause HAVING

Alors que la clause `WHERE` filtre les lignes *avant* le groupement, la clause `HAVING` filtre les groupes *après* que les fonctions d'agrégation ont été appliquées. Elle est utilisée pour spécifier des conditions sur les résultats des fonctions d'agrégation.

- **Exemple** : Trouver les départements ayant plus de 5 employés.

SQL

```
SELECT departement, COUNT(*) AS nombre_employes  
FROM employes  
GROUP BY departement  
HAVING COUNT(*) > 5;
```

Conclusion

La maîtrise du langage SQL, et en particulier des jointures, des fonctions d'agrégation et de la clause `GROUP BY`, est indispensable pour quiconque travaille avec des bases de données relationnelles. Ces concepts permettent d'extraire des informations complexes et de réaliser des analyses de données approfondies, transformant les données brutes en connaissances exploitables.